PARADOJA DEL CUMPLEAÑOS

SIMULACIÓN DE PROBABILIDAD

Instituto Maobits

Arquitectura Mental y Tecnológica para Negocios www.institutomaobits.com

Fundador: Mauricio Chara Hurtado

Empresario del sector de las ciencias de la computación

Perfil Profesional: Empresario informático con más de 10 años de experiencia en el desarrollo de soluciones tecnológicas, formación digital y transformación de procesos mediante pensamiento estructurado, modelamiento y arquitectura lógica.

Lidero el modelo educativo Maobits, enfocado en arquitectura mental aplicada al desarrollo tecnológico y empresarial. Integrando desarrollo de software, producción audiovisual y marketing digital para la transformación empresarial y educativa.

Créditos de Obra Base

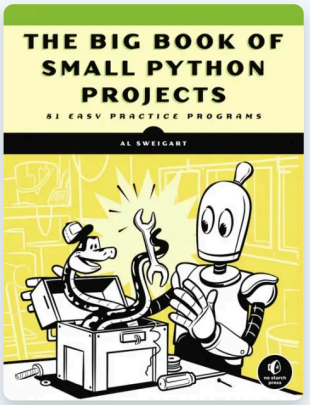
Basado en el proyecto: “Birthday Paradox Simulation”

Libro: "The Big Book of Small Python Projects"

Autor: Al Sweigart

Licencia: Creative Commons

Este proyecto forma parte de una colección curada de más de 80 programas simples en Python diseñados para aprender haciendo.



Descripción del Proyecto

Este programa explora la famosa **"Paradoja del Cumpleaños"**. La paradoja demuestra que en un grupo relativamente pequeño de personas es sorprendentemente probable que al menos dos compartan la misma fecha de cumpleaños.

El programa realiza:

- Generación de cumpleaños aleatorios
- Búsqueda de coincidencias
- Simulación Monte Carlo
- Cálculo aproximado de probabilidades

Este proyecto introduce conceptos de:

- Simulación probabilística y Modelamiento matemático
- Listas y conjuntos
- Comparación de datos y Algoritmos de búsqueda

Explicación Paso a Paso del Código

A continuación, desglosaremos el código fuente de la simulación de la Paradoja del Cumpleaños para entender cómo funciona cada una de sus partes, desde la importación de herramientas hasta la lógica matemática de la simulación Monte Carlo.

1. Importación de Librerías

El programa comienza importando dos módulos estándar de Python que son esenciales para su funcionamiento:

```
# datetime permite trabajar con fechas.
import datetime

# random permite generar valores aleatorios.
import random
```

- `datetime`: Se utiliza para crear y manipular fechas. En este caso, nos ayudará a representar los días del año como fechas reales.
- `random`: Es fundamental para la simulación, ya que nos permite generar números aleatorios (para simular cumpleaños al azar).

2. Configuración del Sistema (Constantes)

Se definen variables en mayúsculas, lo que en programación indica que son **constantes** (valores que no cambiarán durante la ejecución del programa).

```
# Número máximo de cumpleaños que puede generar el usuario.
MAX_CUMPLEANOS = 100

# Número total de simulaciones Monte Carlo.
TOTAL_SIMULACIONES = 100000
```

`MAX_CUMPLEANOS` limita la entrada del usuario para evitar que el programa procese grupos demasiado grandes. `TOTAL_SIMULACIONES` define cuántas veces se repetirá el experimento para calcular la probabilidad (100,000 veces garantiza un resultado estadísticamente muy preciso).

3. Función: Generar Cumpleaños Aleatorios

Esta función se encarga de crear una lista de fechas aleatorias.

```
def generar_cumpleanos(cantidad):
    lista_cumpleanos = []

    for i in range(cantidad):
        # Se define el inicio del año.
        inicio_anio = datetime.date(2001, 1, 1)

        # Se genera un número aleatorio de días dentro del año.
        dias_aleatorios = datetime.timedelta(random.randint(0, 364))

        # Se calcula la fecha final.
        cumpleanos = inicio_anio + dias_aleatorios

        # Se agrega la fecha a la lista.
        lista_cumpleanos.append(cumpleanos)
```

```
    return lista_cumpleanos
```

¿Cómo funciona?

- Recibe un parámetro `cantidad`, que es el número de personas en el grupo.
- Utiliza un año no bisiesto (2001) como base. El año exacto no importa, solo necesitamos 365 días.
- `random.randint(0, 364)` genera un número entre 0 y 364.
- `datetime.timedelta` convierte ese número en una "cantidad de días", que luego se suma al 1 de enero para obtener una fecha aleatoria.
- Finalmente, añade esa fecha a `lista_cumpleanos` y la retorna.

4. Función: Buscar Coincidencia de Cumpleaños

Esta función analiza la lista de cumpleaños generada para ver si al menos dos personas cumplen el mismo día.

```
def buscar_coincidencia(cumpleanos):
    # Si todos los cumpleaños son únicos
    if len(cumpleanos) == len(set(cumpleanos)):
        return None

    # Comparación entre cada cumpleaños
    for indice_a, cumple_a in enumerate(cumpleanos):
        for indice_b, cumple_b in enumerate(cumpleanos[indice_a + 1:]):
            if cumple_a == cumple_b:
                return cumple_a

    return None
```

Optimización inteligente: La línea `len(cumpleanos) == len(set(cumpleanos))` es un truco de Python. Un `set` (conjunto) elimina automáticamente los elementos duplicados. Si la longitud de la lista original es igual a la longitud del conjunto, significa que no hay duplicados, y la función retorna `None` inmediatamente, ahorrando tiempo de procesamiento.

Si hay duplicados, utiliza dos bucles `for` anidados para comparar cada cumpleaños con los demás y retorna la primera fecha repetida que encuentre.

5. Función Principal: Interacción con el Usuario

La función `main()` controla el flujo del programa. Comienza mostrando una introducción y pidiendo datos al usuario.

```
def main():
    print("""Paradoja del Cumpleaños – Simulación de Probabilidad...""")

    MESES = ("Ene", "Feb", "Mar", "Abr", "May", "Jun",
             "Jul", "Ago", "Sep", "Oct", "Nov", "Dic")

    while True:
        print(f"¿Cuántos cumpleaños deseas generar? (Máximo {MAX_CUMPLEANOS})
        respuesta = input("> ")

        if respuesta.isdecimal() and (0 < int(respuesta) <= MAX_CUMPLEANOS):
            cantidad_personas = int(respuesta)
            break
```

Se utiliza un bucle `while True` para validar la entrada. El programa no avanzará hasta que el usuario ingrese un número válido (mayor a 0 y menor o igual a 100). `respuesta.isdecimal()` asegura que el usuario haya escrito números y no letras.

6. Generación y Visualización de la Primera Simulación

Antes de hacer la simulación masiva, el programa hace una demostración visual con el grupo de personas solicitado.

```
    cumpleanos = generar_cumpleanos(cantidad_personas)

    for i, cumple in enumerate(cumpleanos):
        if i != 0:
            print(", ", end="")
        nombre_mes = MESES[cumple.month - 1]
        texto_fecha = f"{nombre_mes} {cumple.day}"
        print(texto_fecha, end="")

    coincidencia = buscar_coincidencia(cumpleanos)
    if coincidencia is not None:
        nombre_mes = MESES[coincidencia.month - 1]
        print(f"\nEn esta simulación, varias personas tienen cumpleaños el {n
    else:
        print("\nEn esta simulación, no hay cumpleaños coincidentes.")
```

Aquí se llama a `generar_cumpleanos` y se imprimen las fechas formateadas (ej. "Mar 15"). Luego, se llama a `buscar_coincidencia` para informar al usuario si en este grupo específico hubo o no un cumpleaños compartido.

7. Simulación Monte Carlo y Cálculo de Probabilidad

Esta es la parte matemática central del proyecto. Una **Simulación Monte Carlo** consiste en repetir un experimento aleatorio muchas veces para encontrar la probabilidad de un evento.

```
    coincidencias = 0
    for i in range(TOTAL_SIMULACIONES):
        # Mostrar progreso cada 10.000 simulaciones
        if i % 10000 == 0:
            print(i, "simulaciones ejecutadas...")

        cumpleanos = generar_cumpleanos(cantidad_personas)
        if buscar_coincidencia(cumpleanos) is not None:
            coincidencias += 1

    # Cálculo de probabilidad
    probabilidad = round(coincidencias / TOTAL_SIMULACIONES * 100, 2)
    print(f"De {TOTAL_SIMULACIONES} simulaciones con grupos de {cantidad_pers
    print(f"Hubo coincidencias de cumpleaños en {coincidencias} casos.")
    print(f"Esto significa que la probabilidad aproximada de coincidencia es:
```

El bucle se ejecuta 100,000 veces. En cada iteración, genera un nuevo grupo de cumpleaños y verifica si hay coincidencias. Si las hay, suma 1 al contador `coincidencias`. Al final, divide las coincidencias entre el total de simulaciones y lo multiplica por 100 para obtener el porcentaje exacto.

8. Ejecución del Programa

Finalmente, el script incluye la condición estándar de Python para ejecutar la función principal.

```
if __name__ == "__main__":
    main()
```

Esta línea asegura que `main()` solo se ejecute si el archivo se corre directamente, y no si es importado como un módulo en otro script de Python. Es una buena práctica en el desarrollo de software.